

y to occur when rebasing the canonical repo
atop the security fork. You will have to resolve these.

Merging security MRs back into the canonical repo

When MRs created in the security repo are merged (into the security repo's main branch), the security and canonical repo will become unsynchronized. Merging MRs from the security fork back into the canonical repo is a manual process.

Each MR in the security repo that a developer wants to incorporate into the canonical repo must be done manually via a new MR in the canonical repo. This procedure is manual so developers can control when these merges are done.

To merge an MR already merged in the security fork main branch into the canonical repo, follow these steps:

From a checked-out canonical repo:

shell

`git fetch` ensure you have the latest changes from the canonical repo.

`git remote add security git@gitlab.com:gitlab-org/security/gitlab-runner.git` add the security repo as a remote, be sure to use the git url.

`git fetch security` fetch the security fork repo references.

`git checkout -b name-of-working-branch origin/main` create a new branch into which you'll cherry-pick commits from the security repo.

`git cherry-pick sha-of-commit-in-security-repo` cherry-pick all commits from the relevant MR from the security repo into your branch in the canonical repo.

Repeat the final step for all commits in the relevant MR, in topographical order, excluding the merge commit. Do not include the MR's merge commit in the cherry-picked commits.

Finally, create an MR in the canonical repo from this branch as usual.

Notes:

1. If the security fork becomes too out of date with the canonical repo, merge conflicts are likely when

cherry-picking the commits. You will have to resolve them.

2. You should manually synchronize the security repo as described above immediate after the MR is merged into the canonical main.

3. It is not the aim of these instruction to completely synchronize the security and canonical repos in both directions.

Full synchronization will occur as a byproduct of merging all MRs from the security repo into the canonical repo. It

is up to the developers' discretion when this happens for each MR.

Issue Health Status Definitions

- On Track - We are confident this issue will be completed and live for the current milestone. It is all downhill from here.
- Needs Attention - There are concerns, new complexity, or unanswered questions that if left unattended will result in the issue missing its targeted release. Collaboration needed to get back On Track within the week.
 - If you are moving an item into this status please mention individuals in the issue you believe can help out in order to unstuck the item so that it can get back to an On Track status.
- At Risk - The issue in its current state will not make the planned release and immediate action is needed to get it back to On Track today.
 - If you are moving an item into this status please consider posting in a relevant team channel in slack. Try to include anything that can be done to unstick the item so that it can get back to an On Track status in your message.
 - Note: It is possible that there is nothing to be done that can get the item back on track in the current milestone. If that is the case please let your manager know as soon as you are aware of this.

Async Issue progress updates

When an engineer is actively working (workflow of ~workflow::"In dev" or further right on current milestone) on an issue they will periodically leave status updates as top-level comments in the issue. The status comment should include the updated health status, any blockers, notes on what was done, if review has started, and anything else the engineer feels is beneficial. If there are multiple people working on it also include whether this is a front end or back end update. An update for each of MR associated with the issue should be included in the update comment. Engineers should also update the health status of the issue at this time.

This update need not adhere to a particular format. Some ideas for formats:

markdown

Health status: (On track|Needs attention|At risk)

Notes: (Share what needs to be shared specially when the issue needs attention or is at risk)

markdown

Health status: (On track|Needs attention|At risk)

What's left to be done:

What's blocking: (probably empty when on track)

markdown

Update <date>

Health status: (On track|Needs attention|At risk)

What's left to be done:

MRs

1. !MyMR1

1. !MyMR2

1. !MyMR3

There are several benefits to this approach:

- Team members can better identify what they can do to help the issue move along the board
- Creates an opening for other engineers to engage and collaborate if they have ideas
- Leaving a status update is a good prompt to ask questions and start a discussion
- The wider GitLab community can more easily follow along with product development
- A history of the roadblocks the issue encountered is readily available in case of retrospection
- Product and Engineering managers are more easily able to keep informed of the progress of work

Some notes/suggestions:

- We typically expect engineers to leave at least one status update per week, barring special circumstances
- Ideally status updates are made at a logical part of an engineers workflow, to minimize disruption
- It is not necessary that the updates happen at the same time/day each week
- Generally when there is a logical time to leave an update, that is the best time
- Engineers are encouraged to use these updates as a place to collect some technical notes and thoughts or "think out loud" as they work through an issue

How to work with us

On issues

Issues worked on by the Runner group a group label of `~group::runner`. Issues that contribute to the verify stage of the DevOps toolchain have the `~devops::verify` label.

Get our attention

GitLab.com: [@gitlab-com/runner-group](https://gitlab.com/runner-group)

Slack: [grunner](#)

Code review

Our code review process follows the general process where you choose a reviewer (usually not a maintainer) and then send it over to a maintainer for the final review.

Current maintainers are members of the `runner-maintainers` group.
Current reviewers are members of the `runner-group` group.

Runner PM and engineering pre and post-sales process for runner scaling and configuration deep dives

As part of the pre-sales and post-sales engagement, your customer may have in-depth questions regarding topics such as GitLab Runner configuration, autoscaling options, how concurrency works, distributing the CI jobs workload, monitoring runners, and so on. The goal of the process below is to enable the runner team to be as efficient as possible in providing the level of support that our sales team and customers require.

Step 1

- Start with the current documentation page on scaling a fleet of runners.

Step 2

- Open an issue in the customer collaboration project and capture the specific configuration questions that the customer has. The purpose of the issue is to address some questions async if possible and finalize the agenda for any follow up synch calls . It also allows us to identify if we need to invite a specific engineer to the customer call. Example issue.

Step 3

- As needed, schedule the sync call with the customer and the Runner PM. The Runner PM will determine if other runner engineers will be included on the call.

Team Resources

See dedicated page.

SECTION: engineering/devops/ops/verify/runner/jtbd.md

Overview

The goal of this page is to create, share and iterate on the Jobs to be Done (JTBD) and their corresponding job statements for the Runner group. Our goal is to utilize the JTBD framework to better understand our buyers' and users' needs.

Goals

Utilize JTBD and job statements to:

- Understand our users' motivations.
- Validate identified use cases and solutions.
- Continuously test and iterate features to ensure we are meeting our customers' and users' needs.
- Create a transparent view for our stakeholders into the current and future state of the product.

Challenges

- DevOps teams may be managing hundreds of Runners, which may also mean offering different

virtual machine configurations and sizes, which adds operational complexity. JTBDs will help us hone in the complex tasks these users need to successfully use our tools.

- Runner's strategy must consider and balance the needs of both self-managed and GitLab.com product offerings.
- Configuring and managing runners are crucial steps in the continuous integration path, but today first-class enterprise management of runners experience is not easily accessible to users. Bringing this capability to a higher level can increase our reach and growth.

JTBD

{{% product/jtbd "Runner" %}}

SECTION: engineering/devops/ops/verify/runner/project-plans.md

Autoscaling Provider for GitLab Runner to replace Docker Machine

Runner Team DRI: Arran Walker

Description: This project's scope is to replace the current autoscaling technology, Docker Machine, used for the GitLab SaaS Shared Runners. To view the complete implementation plan please visit the parent epic that is currently tracking this work.

Week of 2023-09-18

Goals

- ~~] fleeting: [Log AWS Autoscaling Activity~~
 - Implementation revealed questions about whether we really want/need this.
- X] fleeting: [Add Shutdown func to API
- X] fleeting: [Add public provisioning integration tests that plugins can use

Week of 2023-09-25

Goals

- X] fleeting-plugin-aws: [AWS plugin integration tests
- X] taskscaler: [Integration tests

Week of 2023-10-02

Goals

- X] fleeting-plugin-googlecompute: [GCP plugin integration tests
- X] runner: [Taskscaler-based executor integration tests

Week of 2023-10-09

Goals

- X] fleeting-plugin-googlecompute: [GCP plugin integration tests
- X] runner: [Taskscaler-based executor integration tests
-] fleeting-plugin-googlecompute: [Unit tests
- X] runner-incept: [End-to-End test runner manager in GCE
- X] fleeting-plugin-googlecompute: [Recommend minimum IAM permissions for google compute plugin

Week of 2023-10-16

Goals

- X] fleeting-plugin-azure: [Add README / configuration options
-] fleeting-plugin-azure: [Unit tests
-] fleeting-plugin-azure: [Integration tests
-] taskscaler: [Implement Acquisition.WithContext

Dedicated SaaS Runners for GitLab Dedicated

Runner Team DRI: Joseph Burnett

Slack Channel: fhosedrunners

Description: A dedicated runner is a runner that would only be registered to a specific project, group, or instance and not be shared with other users.

With this project GitLab will spin up dedicated runner resources within the Dedicated cloud account.

The project plan tracking this work can be found here.

Iteration 16.5 (ending 2023-10-19)

- [] <https://gitlab.com/gitlab-org/ci-cd/runner-tools/grit/-/issues/14+s>
- [] <https://gitlab.com/gitlab-org/ci-cd/shared-runners/infrastructure/-/issues/87+s>

Week of 2023-09-25

- [X] Verify Runner SaaS delivery timeline meets the requirements of Environment Automation.
It does: <https://gitlab.com/gitlab-com/gl-infra/gitlab-dedicated/team/-/issues/2825note1586596202>

Week of 2023-10-02

Goals

- [X] Consolidate all work under a single sub-epic.
Update issue and epic structure to reflect our agreement to deliver infrastructure-as-a-library (GRIT) and for Environment Automation to operate the runners themselves.

Week of 2023-10-09

Clarified epic and issue structure with Dedicated (thread).

New project plan for Runner side to replace previous epic: <https://gitlab.com/gitlab-org/ci->

cd/shared-runners/infrastructure/-/issues/158

New GRIT sub-epic to track all library work for this use-case:

<https://gitlab.com/groups/gitlab-org/ci-cd/runner-tools/-/epics/2>

Goals

- [X] Create a provisional Docker capable Linux AMI
- X] Update [dev environment to support Linux.

Iteration 16.6 (ending 2023-11-17)

Goals

- [X] Complete test template working end-to-end
- [X] Complete prod template working end-to-end
- [] Unit tests (in progress: <https://gitlab.com/gitlab-org/ci-cd/runner-tools/grit/-/mergerequests/6>)
- [x] End-to-End test (in review: <https://gitlab.com/gitlab-org/ci-cd/runner-tools/grit/-/mergerequests/8>)
- [x] Demo and launch video (<https://www.youtube.com/watch?v=KeOuXN-nXM>)

Week of 2023-11-20

Completed end-to-end functionality for both the test template and prod template.

End-to-end testing has been added and unit tests are in-progress as we decide a reusable approach for all unit test cases.

The demo video for GRIT beta prod was recorded, demonstrating the latest state of GRIT using the prod template.

Iteration Goals

- [] Unit tests (in progress: <https://gitlab.com/gitlab-org/ci-cd/runner-tools/grit/-/mergerequests/6>)
- [] Allow users to bring your own VPC (<https://gitlab.com/gitlab-org/ci-cd/runner-tools/grit/-/issues/35>)
- [] Update READMEs to match latest changes (<https://gitlab.com/gitlab-org/ci-cd/runner-tools/grit/-/issues/36>)

Week of 2023-11-27

If you haven't had a chance, please check out the demo video for GRIT beta prod, which demonstrates the latest state of GRIT using the prod template.

Progress was made last week on adding unit tests, and the only blocker appears to be an issue that is breaking e2e tests in both the unit test branch and the master branch. We will investigate and get the test passing this week.

We will also update READMEs to reflect the recent changes and refactors as well as begin adding granularity to VPC configuration in the prod module.

Goals

- [x] Add Unit tests (in progress: https://gitlab.com/gitlab-org/ci-cd/runner-tools/grit/-/merge_requests/6)
- x] Begin issue: [Allow users to bring your own VPC
- x] [Fix and improve E2E tests

Week of 2023-12-04

We investigated the failing E2E tests and discovered the issue was caused by changes to log levels in GitLab Runner. The E2E tests were fixed and unit tests merged. I began updating the README and discovered our refactors have broken some configurations that involve GCP. These configurations predated our unit and e2e testing. I added an issue and put up an MR to fix the broken configurations and add test coverage. Our current focus is on AWS for Dedicated Runners, so my aim with the fixes for GCP configs in the README is to cover only those README cases at this time and follow up on more thorough GCP tests when we move our focus to GCP at a later date. We also began discussions on how to customize VPCs and subnets in Dedicated Runners.

Goals

-] [Fix broken README configurations
-] [Update READMEs to match latest changes
-] [Allow users to bring their own VPC

Runner Fleet Dashboard

Runner Team DRI: Vladimir Shushlin
Slack Channel: frunnerfleetmanagement

Description: Operators of self-managed runner fleets need, at a glance, observability or, more specifically, the ability to quickly answer critical questions about their Runner Fleet infrastructure.

Providing actionable insights in the Runner Fleet Dashboard equips GitLab Runner Fleets operators with the tools they need to ensure that developers in their organization can consistently and efficiently run CI/CD jobs at scale. The answers to questions such as how fast will CI/CD jobs start, are our CI/CD jobs waiting in a queue, are there are performance or other problems with the CI/CD job environment will be readily available in the Runner Fleet Dashboard.

The result is improved developer efficiency, reduced costs, and excellent customer experience for the development teams that rely on the CI/CD build infrastructure.

Week of 2023-10-02

Goals

- [x] Enable ClickHouse connection on Staging
- x] Enable [Runner Dashboard on Staging
-] Enable [CI data ingestion on Staging

- x] Enable [ClickHouse part of the dashboard on Staging

Summary

We got everything working on Staging, but discovered two bugs in data ingestion:

1. duplicating some data
1. not handling removed cibuilds

So we disabled data ingestion, and are working on fixing it.

We also reviewed the bigger epic, and decided that nothing blocks us from enabling the dashboard for everyone without ClickHouse part of data.

There are ongoing discussions on how to use this ci analytics architecture in other features.

Week of 2023-10-09

Continuing to focus on issues found last week with the goal of re-enabling dashboard on staging and production.

Goals

- x] Fix [duplication bug
- x] Fix [removed cibuilds bug
- x] Rollout and remove [runnersdashboard feature flag thus enabling Runner Dashboard for everyone
- x] Re-enable [CI data ingestion on Staging and finish it
- x] [Add clickhouse credentials for Production
- x] [Add support for clickhouse in Omnibus - that will be required for tests on self-managed as well as using ClickHouse from rails console Staging/Production
- x] [Fix "always integer" bug

Summary

1. We got Runner dashboard enabled on both staging and production including ClickHouse!
2. Decided not to release dashboard in 16.5. We want to test it a bit better and prepare a proper release post.
3. Omnibus part stuck in review a bit.
4. During the data ingestion we discovered that we duplicate some data, it needs investigation

Week of 2023-10-16

With production rollout complete we now will focus on delivering the dashboard(including Clickhouse) to self-managed.

We also have half of the usual backend capacity throughout the 16.6 due to planned time off.

Goals

- x] Create [closed-beta self-managed rollout plan

- x] Finish [omnibus support
- x] [Enable dashboard for self-managed and release in 16.6
- x] (stretch) Get a working PoC for [Clickhouse migrations support

Weeks of 2023-10-23 - 2023-11-20

We released the dashboard for self-managed in 16.6.

And were working delivering ClickHouse-powered part of the dashboard to self-managed.

Goals

- x] Write [the documentation for setting-up the ClickHouse and the dashboard on self-managed.
- x] Implement [the basic support for ClickHouse migrations
- x] Debug [the duplicates issue on gitlab.com

Week of 2023-11-27

With the basic migrations support in place, we're now working on running those migrations automatically

during the normal GitLab upgrade process for omnibus and gitlab chart.

We're also working on fixing the duplicates issue.

Goals

- x] Fix [the duplicates issue.
-] [Run ClickHouse migrations automatically during omnibus upgrade - get it working and in review
-] [Implement exclusive lease to prevent parallel execution of migrations

SECTION: engineering/devops/ops/verify/runner/risk-map.md

Overview

The goal of this page is to create, share and iterate the Risk Map for the Runner team.

Goals

Utilise the Risk Map as a tool to:

- Understand the risks the team faces
- Increase transparency on mitigation plans
- Effectively allocate limited resources
- Collaborate strategically in improving Quality

General Risk Map

Map key

- Impact - what happens if the risk is not mitigated or eliminated
- Impact level - Rate 1 (LOW) to 5 (HIGH)
- Probability - Rate 1 (LOW) to 5 (HIGH)
- Priority - Impact x Probability. Address highest score first.
- Mitigation - what could be done to lower the impact or probability

Risk Area	Risk Description			Impact
Impact Level	Probability	Priority	Mitigation	
Team/Stability	Burn out	Low productivity and attrition		
Minimise overloading and blockers				
Team/Scaling	Inefficient team member onboarding			Prolonged
low productivity		Clear onboarding guidance and prioritisation		
Team/Expertise	Concentration of knowledge			
	Documenting process and knowledge			
Quality/Coverage	Uncertain test coverage			Escaped
bugs		Test coverage analysis and coverage automation		
Quality/Coverage	Sufficient test coverage exercising binaries across supported compute architectures and OSes	Escaped bugs, damage to reputation by inability to claim support due to lack of test coverage		Integration-level test environment and respective test framework
Quality/Coverage	Sufficient test coverage exercising released images	Escaped bugs, damage to reputation due to inability to claim releases were tested		Integration-level test environment and respective test framework
Quality/Infrastructure	Ability to effectively test at release			Escaped
bugs		Reference platforms and standard test harness		
Feature/Dependencies	Bugs in third party dependencies			Bugs
triage, escaped bugs, Failure to execute pipelines		Sufficient test coverage against latest supported version		
Feature/Compatibility	Changes in third party dependencies			Bugs
triage, escaped bugs, Failure to execute pipelines		Testing against multiple dependency versions		
Feature/Function	Functional requirements not met for teams at scale			Low
customer satisfaction for key customers				
Team/Workload	Toil work	Small tasks that should take a few minutes take hours, putting a backlog on reviews/deliverables		
Team/Scaling	Slow pipelines		Take a long time to get feedback on a pipeline and for maintainers to merge something	
Feature/Delivery	Technical debt		When we have so much technical debt it's hard to deliver a feature on time.	
Feature/Delivery	Slow deployment process		Context switching on a feature that you merged weeks ago	
Feature/Dependencies	Not updating 3rd party code		Result in bugs or slow feature delivery since we have to update a bunch of dependencies first	
Observability	Lack of standard on logging	When debugging a problem in production it can be hard to shift through logs if you can't see what is going on in the application		

|
| Observability | Metrics/Dashboards | Hard to debug and
understand what is going with production | |
| Feature/Compatibility | Insufficient testing on supported cloud platforms | Inability to
claim compatibility and feature failures on customer runners | | | Integration-level test
environment and framework |

SECTION: engineering/devops/ops/verify/runner/user-stories.md

User stories

We use user stories at GitLab to guide our solutions as we create design deliverables. User stories are different to JTBDs, but still relate to one another and are important within the product development process. User stories are focused on a persona, the tools they are using, and how and why they are completing the job at hand.

Runner Fleet user stories mapping

All of our user stories map back to our Runner JTBDs.

Sub-Job: managing the execution of CI jobs

- As a Platform Engineer who is administering runners for a GitLab instance or group, I need to know when a job executor is compromised, so I can prevent vulnerabilities from infiltrating my product.
- As a Platform Engineer who is administering runners for a GitLab instance or group, I need to perform general administrative functions as quickly and efficiently as possible, so that I enable my team to continue their daily work.
- As a Platform Engineer who is administering runners for a GitLab instance or group, I need to determine how many runners are out of date by X versions, so that I can help with compliance enforcement.
- As a Platform Engineer who is administering runners for a GitLab instance or group, I need to specify the automatic removal of X runners that have not contacted the GitLab instance for X period of time, so that I can reduce the overhead of maintaining a large fleet of runners.
- As a Platform Engineer who is administering runners for a GitLab instance or group, I need to validate that a team hosting their own runners is on an approved version, so that I can validate it is compliant with my organization.
- As a Software Engineer who is running a CI job, I need to know which of my runners are used for my own development purposes, so I can delete them if they are no longer useful.

Sub-Job: troubleshooting CI jobs`

- As a Platform Engineer who is helping a developer or team troubleshoot issues with a CI job, I need to determine who registered the runner of that CI job, so I can quickly troubleshoot and resolve the issue.
- As a Platform Engineer who is helping a developer or team troubleshoot issues with a CI job, I need to determine which group the runner is associated with, so that I can quickly troubleshoot and resolve the issue.

- As a Platform Engineer who is checking on CI jobs' performance for an instance, I need to see pending and running jobs, so that I can quickly determine if there are performance issues with that runner and the underlying host system or platform.

SECTION: engineering/devops/ops/verify/runner/team-resources/index.md

Overview

The goal of this page is to document resources needed for day-to-day work within the Runner group.

Good bookmarks

- Team Handbook
- Internal Engineering Handbook
- Runner SaaS HQ issue
- Public Runner Docs
- Public Development Docs
- Runner Runbooks
- GitLab.com Triage (for situational awareness)
- Blueprints (search for runner)

Metrics and logs

- Dashboards
 - Runner Service Overview
 - Additional dashboards can be found in the dropdowns along the top bar:
- Metrics
 - Runner Metrics
- Logs
 - Runner Logs (filter by shard)
 - You can find a list of shards in the dropdown along the top bar of any service dashboard:

Internal tools

Merge Request Bot

For
gitlab-org/gitlab-runner
we have the Merge Request Bot enabled which posts
comments for community contributions.
This is configured via Merge Request webhook events.

- Application code
- CloudRun deployment
- Logs

Developing / Testing for Windows

Our development docs for Windows suggest using Vagrant and Virtualbox.

But the easiest way to get started is just to create a Google Compute Engine Windows instance and RDP into it.

Create an instance from this magical image.

Supported versions

We support some pretty old versions of Windows because they are LTSC

Third-party infrastructure

Testing on IBM Z/OS

To facilitate testing the s390x architecture artifacts, a Z/OS VM is available to GitLab team members.

Logging in

1. In 1Password,
under the Verify vault, download the zOS login - gitlabkey02.pem file.
1. From the zOS login entry in the same vault, take note of the user and address fields.
1. SSH into the Z/OS VM:

shell

```
ssh -i "zOS login - gitlabkey02.pem" <user>@<address>
```

Note: You'll be requested the password to unlock the .pem file. Enter the password attached to the zOS login - gitlabkey02.pem entry.

Testing helper image

Assuming you want to test a prebuilt-s390x.tar.xz image produced by a CI/CD pipeline, and already have the .pem file from the previous point, the steps would be the following:

1. Copy the prebuilt-s390x.tar.xz file to the Z/OS VM:

shell script

```
scp -i "zOS login - gitlabkey02.pem" prebuilt-s390x.tar.xz <user>@<address>:/home/ubuntu/
```

Note: You'll be requested the password to unlock the .pem file. Enter the password attached to the zOS login - gitlabkey02.pem entry.

1. SSH into the VM:

```
shell
ssh -i "zOS login - gitlabkey02.pem" <user>@<address>
```

1. Import the image and run it:

```
shell
sudo docker import ./prebuilt-s390x.tar.xz gitlab/gitlab-runner-helper:s390x-dev
sudo docker run -it gitlab/gitlab-runner-helper:s390x-dev bash
gitlab-runner-helper help
```

Accessing Mac Runner AWS environments

GitLab SaaS Mac Runners are running on AWS.

We have production, staging, team sandbox and individual sandbox environments.

An individual sandbox can be created via [Hackystack(<https://gitlabsandbox.cloud/cloud>)].

Be sure to keep an eye on unused resources to reduce cost -- oh-my-cost can help.

We also have a team sandbox in Hackystack which is used to host our Mac Job Image builder instance.

Access to the team sandbox can be acquired via access request.

Within the team sandbox is also a role which has access to the staging and production Mac environments.

Access Mac Runner Staging

From the team sandbox, activate a role named engdevverifyrunner with the account ID 251165465090 (staging).

Access Mac Runner Production

From the team sandbox, activate a role named engdevverifyrunner with the account ID 215928322474 (production).

Load Testing

The group gitlab-runner-stress has a suite of tools for stress testing a GitLab and Runner instance.

Our canonical benchmark for Mac Runners is XcodeBenchmark (our fork).

Runner Vending Machine (AWS Cloud Formation Templates)

The Partner Solution group maintains a curated collection of AWS Cloud Formation Templates for deploying Runner in AWS called the "Runner Vending Machine".

We should keep them in the loop as we change how Runner works and is deployed so these templates can stay up-to-date.

Our point-of-contact is DarwinJS.

Secrets

How we manage secrets for Runner and how they get into the right place is a whole thing. This needs documenting: <https://gitlab.com/gitlab-org/gitlab-runner/-/issues/29823>.

SECTION: engineering/engineering-comms/_index.md

Communication

GitLab Engineering values clear, concise, transparent, asynchronous, and frequent communication. Here are our most important modes of communication:

As part of a fully-distributed organization such as GitLab, it is important to stay informed about engineering-led initiatives.

We employ multimodal communication, which describes the minimum set of communication channels we'll broadcast to.

The Engineering Division has a Google Group, engineering@gitlab.com (internal only), that all members of the division should become members as part of the onboarding process. If this is not the case for you, reach out to your manager. As GitLab, the company, primarily communicates via Slack, use this list mainly for Access Control to Google Drive/Docs/Sheets/Slides.

GitLab Groups/Projects/MRs

- Engineering Division group: This group may contain both public and private sub-groups and projects of various engineering teams.
- CTO Leadership Team issue tracker: Private project that is accessible to all GitLab team members, issues are internal to the company. The tracker hosts topics and initiatives important to the CTO Leadership Team.
- Engineering Division non-product issue tracker: Public project that hosts non-product topics and initiatives important to the Engineering division.
- GitLab, the product, issue tracker: Raise issues pertaining to GitLab product development.
- Start with a merge request: The most effective way to make a change to the company is to make a proposal in the form of a merge request to the handbook and assign it to the DRI.

Monitoring dashboards

- Monitoring dashboards: When we look to achieve a desired outcome, we must be able to quantify and measure progress towards that goal. By making the initial analysis that drives decision making repeatable, we continuously sanity check our initial assumptions along that journey.

Synchronous meetings

- Engineering All-Hands.
- CTO Office Hours: Each week the CTO holds open office hours on Zoom for questions, feedback, and handbook changes. Check the CTO's calendar and look for EMEA and APAC-friendly time slots.
- Principal+ Engineering Demo: this call is an opportunity for sync discussions between Principal+ ICs across the Engineering Division to highlight current ongoing efforts underway

in the groups they support.

- Focuses for the call include:
 - Technical roadmap planning: dependencies, cross-departmental upstream requirements, opportunities for downstream improvements based on current projects. Future initiatives.
 - Opportunities for cross-department alignment, reuse of tooling, modules, processes, standards, technology, documentation.
 - Discussions on improving the career development resources for Engineering ICs at all levels
- All team members are welcome to join the call, but the emphasis is on Principal+ ICs in the Engineering Division to present and discuss the work they're focused on, the problems they're experiencing, and solutions they're considering.
- The call is recorded and available in Google Drive.

Asynchronous updates

Engineering Week-in-Review (EWIR) document

Every week a reminder is sent to the engineering-fyi Slack channel to populate and read the latest updates. The agenda is internal only, please search in Google Drive for Engineering Week-in-Review (internal only)

Slack

engineering-fyi

engineering-fyi is used for large scale announcements and to drive views of the engineering week in review document. Everyone can contribute an announcement as long as the following criteria are met:

- The announcement provides a link back to the appropriate item
 - The announcement involves a majority of one or more departments
 - The announcement is NOT to be company wide and does not need amplification for the department.
- In this situation use company-fyi

The posting model here is one of trusting judgement of the individual making the announcement. You do not need to ask for permission to post.

Sending status updates to engineering-fyi

If you'd like to send status updates to engineering-fyi, please following the following process:

1. For the top level program your reporting on, in the main epic create an issue named `${Program}` Status Updates and populate the description with This issue contains status updates for `${Program}`.
1. In that issue, leave a "threaded" comment with a header `${Program}` Status Update - YYYY-MM-DD to make it clear that the status can be commented on
1. Populate the content of this comment with the update corresponding to that date
1. When you post to engineering-fyi sharing this issue, please include 2-3 sentences that

include the major updates on what was completed and what's blocked or at risk. Please @ tag relevant stakeholders as needed.

1. If this update is the final update for the program, please close the issue after all relevant discussion has taken place.

DevOps Slack Channels

There are primarily two Slack channels which developers may be called upon to assist the production team

when something appears to be amiss with GitLab.com:

- 1. backend: For backend-related issues (e.g. error 500s, high database load, etc.)
- 1. frontend: For frontend-related issues (e.g. JavaScript errors, buttons not working, etc.)

Treat questions or requests from production team for immediate urgency with high priority.

Other Slack channels that are Engineering focused and are good sources of information

- cto
- development
- production
- incidents
- sdeveloperexperience
- supportgitlab-com
- supportself-managed
- splatforms

There is no requirement to join all of these channels. It is up to the person sharing to ensure that the same message is shared across all channels. Ideally, this message should be a one sentence summary with a link to an issue to allow for a single source of truth for any feedback.

SECTION: engineering/engineering-comms/engineering-poster-boards.md

Background

At the FY25 GitLab Summit the Engineering Division held a poster session. This session was well received and several folks recorded videos going over the posters after the Summit, which can be viewed on the GitLab Unfiltered Summit Poster Session Playlist. The team would like to continue doing this in a virtual manner, so that team members and their work are showcased on a regular basis.

Process

The goal is to be a low lift for participating team members. The process to follow is:

- 1. Create an issue to track the overall work using this template.
- 1. Create a virtual poster board using Virutal Poster Session template - CHANGE FILENAME and an

async agenda for the Q&A session

1. Both can be copied by clicking on File -> Make a copy.

1. Store these in a new folder in the Engineering Poster Boards folder in the Engineering Shared Drive

1. Ensure the Q&A agenda is editable by anyone at GitLab.

1. Record a 2-minute overview of the poster - actual time can vary, but we wouldn't expect to have a long recording/presentation. Add this to the Engineering Poster Boards GitLab Unfiltered playlist (to be created).

1. Host a single Q&A session. The agenda can be pre-populated by people who cannot attend. These would be recorded and shared with the team.

- Reach out in eba-team and tag the Engineering EBAs so they can find a suitable time.

Please include the links to the agenda, poster board, video, issue, and any other relevant assets when reaching out. The EBAs will work to find a one week period where we can do multiple poster sessions, one per day, to create engagement. The EBAs will share the week with the presenters and slot in a time that works for them and the team and send an invite from the CTOs calendar including engineering@gitlab.com. The EBAs will ensure to include the shared links for the relevant agenda, poster board, video, and issue in the invite.

SECTION: engineering/error-budgets/_index.md

As part of our strategy to reinforce GitLab SaaS as an enterprise grade platform ready for business critical workloads, GitLab.com has specific Availability and Performance targets.

These targets give our users indication of the platform reliability.

Additionally, GitLab.com Service Level Availability is also a part of our contractual agreement with platform customers. The contract might define a specific target number, and not honouring that agreement may result in financial and reputational burdens.

What are error budgets?

The Google SRE book is generally a recommended read and under the "Motivation for Error Budgets" section, it states:

> The error budget provides a clear, objective metric that determines how unreliable the service is allowed to be within a single quarter. This metric removes the politics from negotiations between the SREs and the product developers when deciding how much risk to allow.

This is the goal we are striving for too, while also acknowledging that in order to arrive at the same level of sophistication, we need to take into account our specific situation, maturity and additional requirements. Our initial approach will directly tie Error Budget SLO with our existing approach to availability.

Future iterations of our error budgets will seek to further develop the importance of the Product Manager in balancing risk tolerance with feature velocity. The above-mentioned clarity between developers and SRE is achieved by establishing the appropriate measures and targets for each service or area of product. Ultimately this balances the importance of new feature work with the ongoing service expectations of users.

What are the components of error budgets?

Error Budgets first depend on establishing an SLO (Service Level Objective). SLOs are made up of an objective, a SLI (Service Level Indicator), and a timeframe.

- Objective: The desired level of success, noted as a percentage
- SLI: an evaluation used to distinguish number of failed events
- Timeframe: enforcing a recency bias to the SLI

Here is an example of these elements:

- Objective: 99.95%
- SLI: 95th percentile latency of api requests over 5 mins is < 100ms
- Timeframe: previous 28 days

Taken all together, the above example SLO would be: 99.95% of the 95th percentile latency of api requests over 5 mins is < 100ms over the previous 28 days

The Error Budget is then $1 - \text{Objective of the SLO}$, in this case $(1 - .9995 = .0005)$. Using our 28 day timeframe, the "budget" for errors is 20.16 minutes $(.0005 (28 \times 24 \times 60))$

While the above example shows the SLI as a latency measurement, it is important to note that other measurements (such as % errors) are also good elements to use for SLIs.

GitLab's current implementation of Error Budgets is only using some of the above sophistication of SLOs and Error Budgets, but we expect to increase the sophistication in the future. It is expected that the practices of SLOs and Error Budgets evolve to have both the objective and the SLI vary (appropriately) based on the criticality of the service as well as the resiliency of other services and components which depend on it.

Which types of errors are included?

Web requests that result in a 500 status code error are counted. In Sidekiq, jobs that fail due to an unhandled exception are counted.

If a group has custom SLIs, or there's an SLI with a fixed feature category configured in our metrics catalog, then those errors will also be counted.

Engineers can use `Gitlab::ErrorTracking.trackexception`, or other logging, freely without affecting the error budget.

Why are we using error budgets?

GitLab is a complex system that needs to be delivered as a highly available SaaS platform. Over the years, several processes have been introduced to address some of the challenges of maintaining feature delivery velocity while ensuring that the SaaS reliability continues to increase.

The Infradev Process was created to prioritize resolving an issue after an incident or degradation has happened. While the process has proven to be successful, it is event-focused

and event-driven.

The Engineering Allocation Process was created to address long term team efficiency, performance and security items.

The initial iteration of error budgets at GitLab aims to introduce objective data and establish a system that will create greater insight into how individual features are performing over an extended period of time. This can be used by the organization to correctly allocate focus, ensure that the risk is well balanced and that the system as a whole remains healthier for extended periods of time.

Assigning error budgets down to the feature category sets a baseline for specific features, which in turn should ensure alignment on prioritizing what's important for GitLab SaaS.

How do we determine the highest priority improvements?

Each group has a Budget spend attribution section in their Budget detail dashboard that allows them to discover where their budget is being spent.

Both the Budget failures panel and each link in the Failure log links panel are ordered by the number of errors. Prioritising fixing the top offenders in these tables will have the biggest impact on the budget spent.

Let's take look at a simplified violation scenario with different traffic patterns:

Endpoint	Total requests	Slow requests	Apdex ratio	Traffic share	
-----	-----	-----	-----	-----	
Endpoint A	1 000	500	50%	1%	
Endpoint B	99 000	9 000	90%	99%	

Even though Endpoint A has a lower apdex ratio, fixing more violations from Endpoint B will have a greater impact on the error budget overall, as they account for the majority of violations impacting the error budget.

Now let's take a look at a more complex violation scenario:

Endpoint	Total requests	Slow requests	Apdex ratio	Traffic share	
-----	-----	-----	-----	-----	
Endpoint A	100 000	30	99.97%	80%	
Endpoint B	25 000	30	99.88%	20%	

The 30 errors from Endpoint A and Endpoint B both have an identical impact on the error budget. However, it would be better to work on Endpoint B because it has a higher ratio of slow requests / overall requests. Because Endpoint A is already meeting the target of 99.95%, no additional work is required here. 99.97% is a good score and any deliberate improvements here could be considered a premature optimization.

The number of violations for Endpoint B puts it below the apdex

threshold, so if these two endpoints are the top violators we see, we should look into improving Endpoint B.

The Error Budget Policy for GitLab.com

The error budgets process has a few distinct items:

1. Budget stakeholders
1. Budget allocation
1. Budget spend and accounting
1. Communication between the stakeholders

Budget stakeholders

The stakeholders in the Error Budget process are:

1. Stage teams (Product department and the supporting Engineering teams represented on the product categories page)
1. Infrastructure teams (Teams represented on the infrastructure team page)
1. VP of Infrastructure and Infrastructure Leadership
1. VP of Development and VP of Product

Budget allocation

Error budget is calculated based on the availability targets.

With the current target of 99.95% availability, allowed unavailability window is 20 minutes per 28 day period.

We elected to use the 28 day period to match Product reporting methods.

The budget is set on the SaaS platform and is shared between stage and infrastructure teams. Service Availability calculation methodology is covered in details at the [GitLab.com Service Availability page](#).

This includes all Rails Controllers, API Endpoints, Sidekiq workers, and other SLIs defined in the service catalog. This is attributed to groups by defining a feature category. Documentation about feature categorization is available in the developer guide.

The number or complexity of features owned by a team, existing product priorities, or the team size does not influence the budget.

Budget Reporting

On the 4th of each month, the Error Budget Report is delivered for the Budget Spend by Stage Group.

The announcements appear in product, eng-managers, ferrorbudgets and development.

There can be months where many stage groups are over budget because of an underlying infrastructure issue.

When the report is generated, if there are more than 5 groups over budget (where the group's traffic share is >0.1%), the Scalability Group will investigate the increased spend before issuing the report. We will announce in the Slack channels that we are investigating before issuing the report.

The intention is to prevent duplicate investigations by multiple teams.

Stage groups are expected to review the monthly Error Budget Report and comment on any overspend in their feature categories.

Error Budgets in the Engineering Allocation Meeting

Stage groups are not expected to report weekly on their Error Budget spend.

Feature categories with monthly spend above the allocated budget for three consecutive months will be added to the agenda for discussion.

Budget spend(by service)

The current budget spend can be found on the general Service Availability dashboard.

Spent budget is the time (in minutes) during which user facing services have experienced a percentage of errors below the specified threshold and latency is above the specified objectives for the service. The details on how Service Availability is calculated can be found at the [GitLab.com Service Availability](#) page.

The budget spend is currently aggregated at the primary service level.

Details on what contributed to the budget spend can be further found by examining the raised incidents, and exploring the specific service dashboard (and its resources).

Budget spend (by stage group)

There is an example available with a more detailed look at how this is built.

The current 28 day budget spend can be found on each stage group dashboard. Feature categories for that stage group are rolled up to a single value.

Stage groups can use their dashboards to explore the cause of their budget spend. The process to investigate the budget spend is described in the developer documentation

The formula for calculating availability:

text

the number of operations with a satisfactory apdex + the number of operations without errors
/

the total number of apdex measurements + the total number of operations

This gives us the percentage of operations that completed successfully and is converted to minutes:

text

(1 - stage group availability) (28 24 60)

Apdex and Error Rates are explained in more detail on the handbook page.

Error Budget Spend information is available on the Error Budgets Overview Dashboard in Tableau.

System-wide incidents

System-wide incidents affecting shared services (such as the database or Redis) may have an impact on a team's

Error Budget spend. Since we look at spend over a 28-day period, the impact of these short lived events should be mostly negligible.

If the impact is significant, we can discuss on the Monthly Report if this incident should warrant a manual adjustment to spend.

At this time we are not looking further into automatically discounting system-wide events from group-level error budgets. The team is focused on building a strong foundation for error budgets, with sufficient tuning capability to be relevant for each group.

How to change error budget attribution

Error budget events are attributed to stage groups via feature categorization. To change the feature category for an endpoint, update the endpoint as described in the feature categorization development documentation.

Updates to feature categories only change how future events are mapped to stage groups. Previously reported events will not be retroactively updated.

The Observability team owns keeping the mappings up to date when feature categories are changed in the website repository. When the categories are changed in stages.yml, a scheduled pipeline creates an issue (example issue) on the build board. The issue contains the pipeline link and instructions to follow in the description. The categories need to be synced to two places:

1. The Rails application.
1. The Runbooks repository.

Error budgets for new groups

When a group gets created, the group is added to the stages.yml. In that merge request, the new group will get assigned new or existing feature categories.

When this MR is merged, Scalability receives an automated issue to update the metrics catalog

in our recording infrastructure. This occurs within one week of the new group being added to stages.yml.

When that metrics catalog is updated, the metrics for the relevant feature categories will get attributed to the new stage group. Similar to when the attribution is changed, the change is not retroactive. This means if an existing feature category moves from source code to code review on the 15th, that feature category will contribute to the error budget of source code for the metrics emitted from the 1st to the 15th. The metrics from the 15th onward will be counted towards the error budget for code review.

The same applies when a group is renamed or a stage moves: if a group rename was merged in the metrics catalog on the 15th, the 28 day report will include 15 days of data for the group's old name, while the most recent days will be attributed to the group with the new name.

Contract

Stage groups who have a traffic share of >0.01% in a given month should abide by this contract to balance feature development with reliability development.
The traffic share for a stage group is visible on the monthly Error Budget Report.

Error Budgets should be reviewed monthly as part of the Product Development Timeline.

The balance between feature development and reliability development for a feature category should be as follows:

Monthly Spend (28 days)	Action
-----	-----
<= 20 minutes	Understand your spend - no further action required.
> 20 minutes	Commitment to reliability/availability improvements, feature development is secondary.

Feature categories with monthly spend above the allocated budget for three consecutive months may have additional feature development restrictions put in place.

Stage Groups with different error budgets

Our current contract is 99.95% availability and a 20 minute monthly error budget. However, the following groups have a temporarily adjusted budget based on business needs:

Stage Group	Monthly Spend (28 days)	Business Reason	Review Date
-----	-----	-----	-----
Tenant Scale: Organizations	99.80%	To allow the group to focus on long-term scalability work (Cells) as well as coordinate changes requiring introduction in the next API version. Described in this MR	2025-10-31 (or if total traffic share exceeds 5%)
Plan:Product Planning	99.89%	Due to an issue checking permissions for participants in a comment in an Epic, the check can be computationally heavy with some endpoints taking over 10 seconds to respond. The team is currently working on optimizing it.	2025-08-29
Plan:Project Management	99.89%	Due to an issue checking permissions for participants in a comment in an Epic, the check can be computationally heavy with some endpoints taking over 10 seconds to respond. The team is currently working on optimizing it.	2025-08-29

Exceptions

Temporary exceptions are granted as a means to allow different stakeholders to fulfill higher priority business needs, if it is estimated that the granted exception is not creating additional risk to GitLab.com reliability. Note that exceptions are different from Custom Targets, which set properties on endpoints defining acceptable performance.

Valid reasons for an exception are:

1. Work for improving the error budget is scoped out and fully planned, and funded. Completing the work will take more than a single release month, and while the work is being completed it is expected that the Error Budget will be regularly spent.
1. Work for improving the error budget is scoped out and fully planned, but the work is not currently funded. The stakeholders are in the process of securing the funding, and the Error Budget will be regularly spent until the additional funds are secured.
1. Temporarily, the highest priority is to achieve a significant business goal, and the reliability of GitLab.com is not directly affected. The group is likely to regularly spend the Error Budget while they are focused on this other priority.

Instructions for Requesting an Exception

To request an exception, open an MR and add the stage group to the table above. In the description, supply the following details:

1. Clear description of the problem that is the cause of the budget spend
1. Relevant resources showing that the work is scoped out
1. Target date when the exception must be revisited

Additional Guidance

1. Document the work to be done using Epics and Issues
1. Add detailed descriptions to the Epics and Issues to ensure the work is clearly scoped out.
1. Add a start date and due date to the Epic so that it is transparent how long it will take to complete the work.
1. Add milestones to the issues so it is transparent when the work will be planned.

Provide answers to the following questions:

1. What portion of your team's budget is due to this exception? If you were to remove the offending endpoints covered by this exception would your error budget become green?
1. What is the main contributor to your team's error budget spend? Is that the response time?
1. What does success look like at the closure of referenced epic?

Follow the guidance and instructions above to expedite the approval process.

Assign the MR for approval to:

1. Director of Product or above (of the affected stage group)
 - They are responsible for ensuring that the business need is met, and will need to communicate the change up and down the chain of reporting.

1. Senior Manager of Infrastructure or up

- They are responsible for ensuring that GitLab.com will not be negatively impacted, and will need to communicate the exception up and down the chain of reporting.

Error Budget Improvements

Work relating to error budget improvements should be detailed in an issue.

Please label these issues with `Error Budget

Improvement and the group::` label so they can be tracked in reports.

Error Budget DRIs

| Role | K/PI | Target | Current Tracking Status |

| --- | --- | --- | --- |

| Product Management | Maintaining the Spend of the Error Budget | 20 minutes over 28 days (equivalent to 99.95% availability) | Complete - In Sisense |

| Infrastructure | Setting the Error Budget Minutes and Availability Target | 99.95% (20 minutes over 28 days Error Budget) | Complete - In Grafana |

- For groups with engineering allocations, the responsibility to maintain the spend of error budget is with the development team instead of the product management team.

Current State and Future Intent

Current State

1. Error budgets exist for each feature category and incorporate a standard apdex threshold and error rate.

1. Error budgets are published for stage groups and stages through Grafana and Sisense Dashboards.

1. Contributing factors are explorable through links available on the Grafana Dashboards.

1. Error budgets are included in the Product Prioritization process.

Roadmap

The changes below aim to increase the maturity of the Error Budgets.

1. Increase precision of Error Budget calculations (apdex portion)

Improvements

- Cancelled The SLO targets originally used for Error Budgets are coupled with the alerting used for Infrastructure monitoring. We proposed to use Sisense to be able to set targets by stage group, but this approach was not favoured. We found a method to use separate targets for the Infrastructure monitoring and the Error Budgets, but the decision was taken to keep the targets the same and adjust the default latency threshold for the apdex portion of the Error Budgets (see next item).

- Completed SLI calculations used request duration threshold which was not appropriate for all endpoints. The threshold was increased to 5s on the 21st of Sept and it will take 28 days for the full effect to be shown in the Error Budgets.

- Completed Stage groups will next be enabled to set their own SLI per endpoint by expanding on the configurability of SLI request duration threshold. epic.
- Endpoints that are currently not owned will be attributed to the correct feature category. This will be addressed by
 - Completed using caller information for Sidekiq, and
 - having GraphQL query-to-feature correlation.
- The impact of system-wide outages on Error Budgets should be more clear.
- Provide guidance for PM's who report on both Error Budgets and Service Availability. (Such as Runner and Pages).

Product Development Activities

Product Development teams are encouraged to:

- Continue working on Rapid Action, Infradev, Corrective Actions, Security, and Engineering Allocation issues per our Prioritization guidelines
- Propose SLOs for their endpoints
- Opt-in to using the new apdex calculation methods that use the custom target durations
- Provide further feedback for future improvements to Error Budgets

2. Increase visibility into Error Budgets (error portion)

- Stage groups are provided with error count information. This can be supplemented with further detail by making error information explorable with Sentry.

3. Tune the scope of Error Budgets

- Consider incorporating P1/S1 incidents into the Error Budget Calculation.

More information

- Error Budget AMA
- Understanding Stage Level Error Budget Dashboards
- Setting up recurring Slack updates

SECTION: engineering/error-budgets/custom-targets.md

What are Custom Targets for Error Budgets?

Custom Targets introduce the ability to set tailored Error Budget durations on individual endpoints and jobs, allowing for the accepted performance of different workloads to be defined as appropriate.

There is a legacy apdex threshold used for every endpoint which is currently set to 5 seconds. When a request duration exceeds 5 seconds, that request is considered in violation of this threshold and counts against the apdex portion of the Error Budget. Because this does not cater for the different performance characteristics this component is being replaced with a customizable apdex threshold.

Why do we use Custom Targets for Error Budgets?

Performance expectations vary for different workloads and can form a vital part of a good user experience. It is therefore important to let the teams who work closest with customers define what durations are acceptable for the various endpoints and jobs they own. Performance of specific workloads can also contribute towards a larger, system-wide impact - in this case, the Infrastructure Department may request that target durations are adjusted appropriately.

How to set target durations

Stage groups can follow the developer documentation to set target durations and should ask for a review from a Scalability Team Member on the MR.

The developer documentation includes instructions for how to choose a target duration, how to set the duration, and what considerations should be made.

How are target durations used?

When target durations are set, the system will start recording data for those targets as soon as the MR is deployed to production. This means that any violations or successes that have occurred before the deploy would still be counted towards the error budget for stage groups.

For now, Error budgets will continue to use the legacy SLI using a 5 second target duration until the stage group chooses to opt-in to using the customizable durations. For customizable durations the default threshold (for endpoints without a specific target) is set to 1 second.

There is a dashboard (internal only) available to view which stage groups have opted-in to this method, in addition to showing a comparison of the new apdex ratio (adjusted for custom targets) against the old apdex ratio (based on the 5 second default).

SECTION: engineering/error-budgets/error-budget-by-stage-group-example.md

Error Budgets for Stage Groups (worked example)

Error Budgets consist of two components: Apdex and Error Rate.

Apdex Success Rate: The rate of operations that were successful and performed adequately.

The threshold for 'performed adequately' is different for each service.

This is currently a global threshold per service, but stage groups will soon have the ability to customise this by endpoint.

Error Rate: The rate of operations that had errors.

The developer documentation contains detailed steps for how to check where budget is being spent.

SLI, SLO, SLA?

These are Service Level Indicators, Objectives and Agreements.

An SLA is an agreement that one group has set with another regarding the level of service that will be provided.

We have an SLA with our customers to achieve a certain level of availability each month. Currently this is 99.95%.

We use that agreement to set service-level objectives (SLOs). These are the standards we must meet each month in order to fulfil our agreements.

Finally, the SLI is the indicator we use to determine if we will meet our objective. It is the measure of how our systems are performing.

The SLA is the percentage of time that SLIs met their SLO.

Storing the data to use in calculations

In this section, we talk about Apdex for Web and API endpoints.

Every endpoint is associated with a feature category.

We use this to help with incident response as well as to attribute error budget spend to the right stage group.

For every request, we store log information - including:

- the endpoint being requested
- the feature category that owned the endpoint at the time
- how long it took to generate and serve the response
- the time the request spent querying other resources
- the status code for the response
- (a variety of other things)

The highest granularity of data is stored in the logs, but we can't hold onto this data for long periods because we generate quite a lot of it. We retain log information for 7 days.

We also store metrics for a longer period which are at a lower granularity. One of the items we store a metric for is the response duration.

Because of size constraints we can't store the exact duration for a request in the metrics. Instead we use a histogram with buckets of $[-\text{Inf}, 0.1, 0.25, 0.5, 1.0, 2.5, 5.0, +\text{Inf}]$ which are [defined in the metrics catalog.

When a request takes 0.6s, it would increment the buckets for which it was faster. So $[+\text{Inf}, 5.0, 2.5, 1.0]$ would be incremented.

We also store if the request was faster or slower than the request duration threshold for that

endpoint. This is done using Application SLIs. This allows us to customize SLIs based on the importance to users. For example, the urgency for the railsrequest SLI can be customized to reflect how a user experiences the endpoint.

Metric information

We store metrics in Prometheus as counters.

Apdex

- operation-counter: This counter get incremented for every successful request (no error).
- success-counter: This counter gets incremented for every successful request that was faster than 5s.

Error

- operation-counter: This counter get incremented for every